



Université Mohammed Premier
Faculté Pluridisciplinaire de Nador

Compte Rendu : Système d'Exploitation 1

TDM1-TDM2-TDM3

Nom et Prénom : Hamich Nassira

Filière : Informatique et Intelligence Artificielle (IIA)

Semestre : S3

Enseignant : Khalid El Makkaoui

Année universitaire : 2025/2026

Compte Rendu - TDM n°1

Exercice 1 : Questions de cours :

► 1. Les principales caractéristiques de Linux :

Linux est un système ouvert ; son code source est **libre et accessible à tous**. Il est **portable**, ce qui signifie qu'il peut fonctionner sur plusieurs types d'ordinateurs comme les PC ou les serveurs.

Il est **multi-tâche**, donc capable d'exécuter plusieurs programmes en même temps.

Il est **multi-utilisateurs**, ce qui permet à plusieurs personnes d'utiliser le système en même temps.

Enfin, il est **gratuit**, sauf si l'on paie pour un support technique.

► 2. Le rôle du noyau de Linux :

Le noyau est le cœur du système d'exploitation.

Il s'occupe de la mémoire, en la partageant entre les programmes.

Il **gère** les processus, c'est-à-dire la création, l'exécution et la communication entre eux.

Il **contrôle** le système de fichiers et les périphériques.

Il **gère** aussi les entrées-sorties et les connexions réseau.

Enfin, il fait le lien entre le matériel de l'ordinateur et les logiciels.

► 3. Le rôle du shell :

Le shell est le programme qui comprend et exécute les commandes de l'utilisateur.

Il analyse les commandes, les envoie au noyau, puis affiche les résultats.

C'est donc une interface entre l'utilisateur et le système.

Le shell permet aussi d'automatiser des tâches grâce à des scripts.

► 4. Différence entre virtualisation basée sur l'hôte et virtualisation sur matériel :

Dans **la virtualisation basée sur l'hôte**, un système d'exploitation est déjà installé sur la machine. Le logiciel de virtualisation comme VirtualBox ou VMware Workstation fonctionne à l'intérieur de ce système. Les machines virtuelles dépendent donc de lui, ce qui réduit un peu les performances.

Dans **la virtualisation sur matériel**, l'hyperviseur est installé directement sur le matériel de l'ordinateur, sans système d'exploitation au-dessus.

Exemples : VMware ESXi, Hyper-V ou Xen.

Cette méthode offre de meilleures performances et est surtout utilisée dans les serveurs professionnels.

Exercice 2 : les Commandes d'information système

1. Afficher des informations détaillées sur la distribution Linux installée.

lsb_release : commande qui affiche les informations LSB (Linux Standard Base).

-a : option pour afficher toutes les informations disponibles.

Exemple :

:~\$ lsb_release -a

```
ubuntu@ubuntu:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 24.04.3 LTS
Release:        24.04
Codename:       noble
```

2. Afficher le nom de la machine.

Méthode 1 :

hostname : commande qui affiche le nom d'hôte de la machine (le nom du PC sur le réseau).

Exemple :

:~\$ hostname

```
ubuntu@ubuntu:~$ hostname
ubuntu
```

Méthode 2 :

Uname (Unix Name) : commande qui affiche les informations système.

n (nodename) : option qui affiche le nom du nœud réseau (hostname).

Exemple :

:~\$ uname -n

```
ubuntu@ubuntu:~$ uname -n
ubuntu
```

2. Afficher le nom d'utilisateur actuel.

Méthode 1 :

logname : commande qui affiche le nom de l'utilisateur actuellement connecté.

Exemple :

:~\$ logname

```
ubuntu@ubuntu:~$ logname
ubuntu
```

Méthode 2 :

echo : commande qui affiche du texte ou la valeur de variables.

\$USER : variable d'environnement qui contient le nom de l'utilisateur courant .

Le "\$" indique qu'on veut la **valeur** de la variable (pas le mot <<USER>>).

Exemple :

:~\$ echo \$USER

```
ubuntu@ubuntu:~$ echo $USER
ubuntu
```

4. Afficher la liste des utilisateurs actuellement connectés.

who: commande qui affiche la liste de tous les utilisateurs connectés au système.

Exemple :

```
:~$ who
```

```
ubuntu@ubuntu:~$ who
ubuntu  seat0      2025-11-06 11:53 (login screen)
ubuntu  :0           2025-11-06 11:53 (:0)
```

Exercice 3 : Manipulation des répertoires

1. Vérifions si nous sommes dans le répertoire de connexion.

pwd (Print Working Directory): commande qui affiche le chemin absolu du répertoire courant .

Exemple :

```
:~$ pwd
```

```
ubuntu@ubuntu:~$ pwd
/home/ubuntu
```

3. Dans le répertoire de connexion, nous créons l'arborescence IIA/S3/SEI.

mkdir (Make Directory) : commande qui crée un ou plusieurs répertoires.

cd (Change Directory): commande qui permet de **changer de répertoire** (se déplacer dans l'arborescence des fichiers)

-p (parents): option qui crée tous les répertoires parents nécessaires s'ils n'existent pas déjà .

Méthode 1 :

Avec “-p” : tout est créé en une seule commande

Exemple :

```
:~$ mkdir -p IIA/S3/SEI
```

```
ubuntu@ubuntu:~$ mkdir -p IIA/S3/SEI
```

Le Structure créée :

```
IIA/
├── S3/
│   └── SEI/
```

Méthode 2 :

Sans “-p” : vous devriez créer “IIA”, puis “S3”, puis “SEI” séparément.

Exemple :

```
:~$ mkdir -p IIA
```

```
:~$ mkdir -p IIA/S3
```

```
:~$ mkdir -p IIA/S3/SEI
```

```
ubuntu@ubuntu:~$ mkdir IIA
ubuntu@ubuntu:~$ mkdir IIA/S3
ubuntu@ubuntu:~$ mkdir IIA/S3/SEI
```

Méthode 3 : Combiner mkdir et cd (pas à pas)

```
ubuntu@ubuntu:~$ mkdir IIA
ubuntu@ubuntu:~$ cd IIA
ubuntu@ubuntu:~/IIA$ mkdir S3
ubuntu@ubuntu:~/IIA$ cd S3
ubuntu@ubuntu:~/IIA/S3$ mkdir SEI
```

3. Dans le bureau, nous créons le répertoire *Compte_rendu1*.

mkdir : crée un répertoire .

Desktop : chemin vers le bureau (Desktop) de l'utilisateur.

Compte_rendu1 : nom du répertoire à créer

Exemple :

```
:~$ mkdir Desktop/Compte_rendu1
```

```
ubuntu@ubuntu:~$ mkdir Desktop/Compte_rendu1
```

Le chemin complet créé :

/home/ubuntu/Desktop/Compte_rendu1

4. Déplaçons *Compte_rendu1* vers *SEI*.

mv (Move) : commande qui déplace ou renomme des fichiers/répertoires.

Desktop/Compte_rendu1 : source (ce qu'on veut déplacer)

IIA/S3/SEI : destination (où on veut le déplacer)

Exemple :

```
:~$ mv Desktop/Compte_rendu1 IIA/S3/SEI
```

```
ubuntu@ubuntu:~$ mv Desktop/Compte_rendu1 IIA/S3/SEI
```

Résultat :

Avant : /home/ubuntu/Desktop/Compte_rendu1

Après : /home/ubuntu/IIA/S3/SEI/Compte_rendu1

4. Affichons les répertoires sous forme d'arborescence.

Méthode 1 :

tree : commande qui affiche l'arborescence des répertoires de manière visuelle

IIA : répertoire de départ pour l'affichage

Exemple :

```
:~$ tree IIA
```

```
(linux@DESKTOP-DK89QUP)-[~]
$ tree IIA
IIA
├── S3
│   └── SEI
│       └── Compte_rendu1
4 directories, 0 files
```

Méthode 2 :

ls (List) : liste le contenu des répertoires.

-R (Recursive) : affiche récursivement tous les sous-répertoires.

Exemple :

```
:~$ ls -R IIA
```

```
ubuntu@ubuntu:~$ ls -R IIA
IIA:
S3

IIA/S3:
SEI

IIA/S3/SEI:
Compte_rendu1

IIA/S3/SEI/Compte_rendu1:
```

6 .Supprimons le répertoire IIA.

Méthode 1:

rm (Remove) : commande qui supprime des fichiers ou répertoires.

-r (Recursive) : option obligatoire pour supprimer un répertoire et tout son contenu.

IIA : le nom du répertoire à supprimer.

Exemple :

```
:~$ rm -r IIA/S3/SEI/Compte_rendu1
```

```
ubuntu@ubuntu:~$ rm -r IIA/S3/SEI/Compte_rendu1
```

Méthode 2:

rmdir (Remove Directory): commande qui permet de supprimer des répertoires VIDE uniquement

Exemple :

- avec l'option "-p" :

```
:~$ rmdir -p IIA/S3/SEI/Compte_rendu1
```

```
ubuntu@ubuntu:~$ rmdir -p IIA/S3/SEI/Compte_rendu1
```

- sans l'option "-p" :

```
:~$ rmdir -p IIA/S3/SEI/Compte_rendu1
```

```
:~$ rmdir -p IIA/S3/SEI
```

```
:~$ rmdir -p IIA/S3
```

```
:~$ rmdir -p IIA
```

```
ubuntu@ubuntu:~$ rmdir IIA/S3/SEI/Compte_rendu1
ubuntu@ubuntu:~$ rmdir IIA/S3/SEI
ubuntu@ubuntu:~$ rmdir IIA/S3
ubuntu@ubuntu:~$ rmdir IIA
```

Exercice 4 : Manipulation des fichiers

1. Dans le répertoire de connexion, nous créons les répertoires *TDM1* et *TDM_SEI*.

Méthode 1:

mkdir : crée des répertoires.

TDM1 TDM1_SEI : liste de répertoires séparés par des espaces (crée les deux en même temps).

Exemple :

```
:~$ mkdir TDM1 TDM1_SEI
```

```
ubuntu@ubuntu:~$ mkdir TDM1 TDM1_SEI
```

Résultat :

```
/home/ubuntu/TDM1
```

```
/home/ubuntu/TDM1_SEI
```

Méthode 2: crée un par un .

```
ubuntu@ubuntu:~$ mkdir TDM1
ubuntu@ubuntu:~$ mkdir TDM1_SEI
```

2. nous créons le fichier *TDM1.txt* .

(fichier vide) :

touch : commande qui crée un fichier vide .

TDM1.txt : nom du fichier à créer.

Exemple :

```
:~$ touch TDM1.txt
```

```
ubuntu@ubuntu:~$ touch TDM1.txt
```

(avec contenu) :

cat > : redirige l'entrée standard (ce que vous tapez) vers un fichier.

> : opérateur de redirection qui écrase le fichier s'il existe.

Après avoir tapé votre contenu, Ctrl+D signale la fin de l'entrée

Exemple :

```
:~$ cat > TDM1.txt
```

```
ubuntu@ubuntu:~$ cat > TDM1.txt
ceci est le contenu de mon fichier
```

Remarque :

On peut aussi créer/éditer un fichier avec **nano** (éditeur de texte simple et intuitif), **vi**, **vim** et **gedit**..

Exemple :

```
:~$ nano TDM1.txt
```

```
ubuntu@ubuntu:~$ nano TDM1.txt
```

3. on Duplique le fichier *TDM1.txt* sous le nom *TDM1_SEI.txt*.

cp (Copy) : commande qui copie des fichiers ou répertoires.

TDM1.txt : fichier source (à copier).

TDM1_SEI.txt : fichier destination (la copie).

Exemple :

```
:~$ cp TDM1.txt TDM1_SEI.txt
```

```
ubuntu@ubuntu:~$ cp TDM1.txt TDM1_SEI.txt
```

Résultat :

TDM1.txt → existe

TDM1_SEI.txt → nouveau fichier créé (copie identique)

4. Déplaçons le fichier TDM1_SEI.txt dans le répertoire TDM1_SEI.

mv : déplace (ou renomme) le fichier

TDM1_SEI.txt : fichier source (à déplacer)

TDM1_SEI/ : répertoire destination (le '/' indique que c'est un répertoire).

Exemple :

```
:~$ mv TDM1_SEI.txt TDM1_SEI
```

```
ubuntu@ubuntu:~$ mv TDM1.txt TDM1_SEI
```

Résultat :

Avant : /home/ubuntu/TDM1_SEI.txt

Après : /home/ubuntu/TDM1_SEI/TDM1_SEI.txt

5. on affiche le contenu du répertoire TDM1_SEI en détail.

ls : liste le contenu d'un répertoire

-a (all) : affiche aussi les fichiers cachés (commençant par .)

-l (long format) : affiche les détails (permissions, propriétaire, taille, date)

TDM1_SEI : répertoire à lister.

Exemple :

```
:~$ ls -al TDM1_SEI
```

```
ubuntu@ubuntu:~$ ls -al TDM1_SEI
total 4
drwxrwxr-x  2 ubuntu ubuntu  60 Nov  6 12:45 .
drwxr-x--- 20 ubuntu ubuntu 520 Nov  6 12:45 ..
-rw-rw-r--  1 ubuntu ubuntu  35 Nov  6 12:25 TDM1.txt
```

6. on supprime le fichier TDM1.txt en demandant une confirmation avant la suppression.

rm : supprime un fichier

-i (interactive) : demande confirmation avant de supprimer

TDM1.txt : fichier à supprimer

Exemple :

```
:~$ rm -i TDM1.txt
```

```
ubuntu@ubuntu:~$ rm -i TDM1_SEI/TDM1.txt
rm: remove regular file 'TDM1_SEI/TDM1.txt'? █
```

Tapez y (yes) pour confirmer

Tapez n (no) pour annuler

Compte Rendu - TDM n°2

Exercice 1 :

1- le rôle de la commande cat :

La commande **cat** (concatenate) permet d'afficher, de créer et de concaténer des fichiers.

Exemples :

- Afficher le contenu d'un fichier:

```
ubuntu@ubuntu:~$ cat file.txt
```

- Créer un nouveau fichier :

```
ubuntu@ubuntu:~$ cat > NewFile.txt
```

(tapez le contenu puis Ctrl+D)

- Concaténer plusieurs fichiers :

```
ubuntu@ubuntu:~$ cat file.txt NewFile.txt > file2.txt
```

- Afficher avec numérotation des lignes :

```
ubuntu@ubuntu:~$ cat -n file2.txt
 1  Compte Rendu TDM n2
 2  TP SE
```

2- Soit le fichier "file.txt" contient 100 lignes de texte :

- créer le fichier **file.txt** avec 100 des lignes :

```
ubuntu@ubuntu:~$ cat > file.txt
```

3- un fichier nommé file.txt contenant 100 lignes de texte :

- a- Afficher le nombre de lignes de ce fichier.

```
ubuntu@ubuntu:~$ wc -l file.txt
1 file.txt
```

La commande : **wc** (word count) avec l'option **-l** compte le nombre de lignes.

- b- Afficher le nombre de mots, de lignes et de caractères de fichier file.txt

```
ubuntu@ubuntu:~$ wc file.txt
```

Le Résultat : affiche dans l'ordre : nombre de lignes, nombre de mots, nombre de caractères, nom du fichier.

c-Trier le contenu du fichier en ordre croissant et rediriger le résultat vers un autre fichier.

Méthode 1 :

```
ubuntu@ubuntu:~$ sort file.txt > sortedfile.txt
```

Méthode 2 :

```
ubuntu@ubuntu:~$ sort file.txt -o sortedfile.txt
```

La commande : **sort** trie les lignes par ordre alphabétique/numérique croissant.

d- Afficher, pour chaque ligne d'un fichier, les caractères situés aux positions 1 à 7 et aux positions 10 à 19.

```
ubuntu@ubuntu:~$ cut -c1-7,10-19 file.txt
Compte ndu TDM n2
```

La commande : **cut** avec l'option **-c** (characters) extrait les caractères aux positions spécifiées.

e- Afficher les 7 premières lignes de ce fichier.

```
ubuntu@ubuntu:~$ head -n 7 file.txt
Compte Rendu TDM n2
```

ou :

```
ubuntu@ubuntu:~$ head -7 file.txt
Compte Rendu TDM n2
```

La commande : **head** affiche le début d'un fichier. L'option **-n 7** spécifie 7 lignes.

f- Afficher les 7 dernières lignes de ce fichier.

```
ubuntu@ubuntu:~$ tail -n 7 file.txt
Compte Rendu TDM n2
```

ou :

```
ubuntu@ubuntu:~$ tail -7 file.txt
Compte Rendu TDM n2
```

La commande : **tail** affiche la fin d'un fichier.

g-Rechercher dans le fichier la chaîne de caractères "passwd" et afficher les lignes correspondantes avec leur numéro de ligne.

Exemple 1 :

```
ubuntu@ubuntu:~$ grep -n "passwd" file.txt
```

Exemple 2 :

```
ubuntu@ubuntu:~$ grep -n "TDM" file.txt
1:Compte Rendu TDM n2
```

La commande : **grep** recherche un motif. L'option **-n** affiche les numéros de lignes.

Exercice 2 :

1. Les droits d'accès actuels du fichier **file.txt** sont **rw-rw-r--**.

- ✓ On Ajouter au propriétaire le droit d'exécution (x)
- ✓ On Ajouter aux autres utilisateurs le droit d'écriture (w)

En utilisant la forme numérique et la forme symbolique.

Forme symbolique :

```
ubuntu@ubuntu:~$ chmod u+x,o+w file.txt
```

Forme numérique :

Droits actuels : **rw-rw-r--** = 664

- Propriétaire : rw- = 6
- Groupe : rw- = 6
- Autres : r-- = 4

Droits demandés : **rw-rw-rw-** = 766

- Propriétaire : rwx = 7 (ajout de x)
- Groupe : rw- = 6 (inchangé)
- Autres : rw- = 6 (ajout de w)

```
ubuntu@ubuntu:~$ chmod 766 file.txt
```

2. Utiliser les commandes adéquates pour obtenir exactement l'arborescence suivante :

```
rep
├── file.txt
├── lienPhysique.txt
└── lienSymbolique.txt -> file.txt
```

Commandes :

Créer le répertoire **rep** :

```
ubuntu@ubuntu:~$ mkdir rep
```

Créer le fichier **file.txt** :

```
ubuntu@ubuntu:~$ touch rep/file.txt
```

Créer le lien physique :

```
ubuntu@ubuntu:~$ ln rep/file.txt rep/lienPhysique.txt
```

Créer le lien symbolique :

```
ubuntu@ubuntu:~$ ln -s rep/file.txt rep/lienSymbolique.txt
```

La commande **ln** :

- **ln** crée un lien physique (hard link)
- **ln -s** crée un lien symbolique (soft link)

3. Rechercher tous les fichiers d'extension ".txt" à partir du répertoire courant.

```
ubuntu@ubuntu:~$ find . -name "*.txt"
```

La commande : **find** recherche des fichiers. L'option **-name** filtre par nom avec wildcard *****.

4. Rechercher les fichiers de type lien symbolique dans rep.

```
ubuntu@ubuntu:~$ find rep -type l
rep/lienSymbolique.txt
rep/lienSymbolique.txt
```

La commande : L'option **-type l** recherche uniquement les liens symboliques.

5. Afficher le répertoire rep et ses fichiers sous forme d'arborescence.

```
ubuntu@ubuntu:~$ tree rep
rep
├── file.txt
├── lienPhysique.txt
├── lienSymbolique.txt -> file.txt
└── lienSymbolique.txt -> rep/file.txt

1 directory, 4 files
```

ou sans la commande **tree** :

```
ubuntu@ubuntu:~$ ls -R rep
rep:
file.txt  lienPhysique.txt  lienSymbolique.txt
```

Exercice 3 : Comptage de fichiers dans rep3

Soit le répertoire rep3 contenant plusieurs fichiers et sous-répertoires.

1. Compter le nombre de fichiers ordinaires dans le répertoire rep3 à l'aide des commandes **ls** et **wc** :
(a) en utilisant les tubes anonymes (pipe).

```
ubuntu@ubuntu:~$ ls -l rep3 | grep "^-" | wc -l
0
```

La commande :

- **ls -l** : liste détaillée

- **grep "^-"** : filtre les lignes commençant par - (fichiers ordinaires)
- **wc -l** : compte les lignes
- **|** : tube anonyme qui redirige la sortie d'une commande vers l'entrée de la suivante

(b) en utilisant la redirection vers des fichiers.

```
ubuntu@ubuntu:~$ cat erreurs_wc.txt
ubuntu@ubuntu:~$ ls -l rep3 > liste_temp.txt
ubuntu@ubuntu:~$ grep "^-" liste_temp.txt > files_temp.txt
ubuntu@ubuntu:~$ wc -l files_temp.txt
0 files_temp.txt
ubuntu@ubuntu:~$ rm liste_temp.txt files_temp.txt
```

2. Compter le nombre de sous-répertoires du répertoire rep3 à l'aide des commandes ls et wc, de deux manières différentes, en redirigeant les messages d'erreur de chaque commande vers des fichiers distincts.

Méthode 1 : Avec tubes et redirection d'erreurs

```
ubuntu@ubuntu:~$ ls -l rep3 2> erreurs_ls.txt | grep "^d" 2> erreurs_grep.txt | wc -l 2> erreurs_wc.txt
1
```

Méthode 2 : Avec fichiers intermédiaires

```
ubuntu@ubuntu:~$ ls -l rep3 > liste.txt 2> erreurs_ls.txt
ubuntu@ubuntu:~$ grep "^d" liste.txt > rep3.txt 2> erreurs_grep.txt
ubuntu@ubuntu:~$ wc -l rep3.txt 2> erreurs_wc.txt
1 rep3.txt
ubuntu@ubuntu:~$ rm liste.txt rep3.txt
```

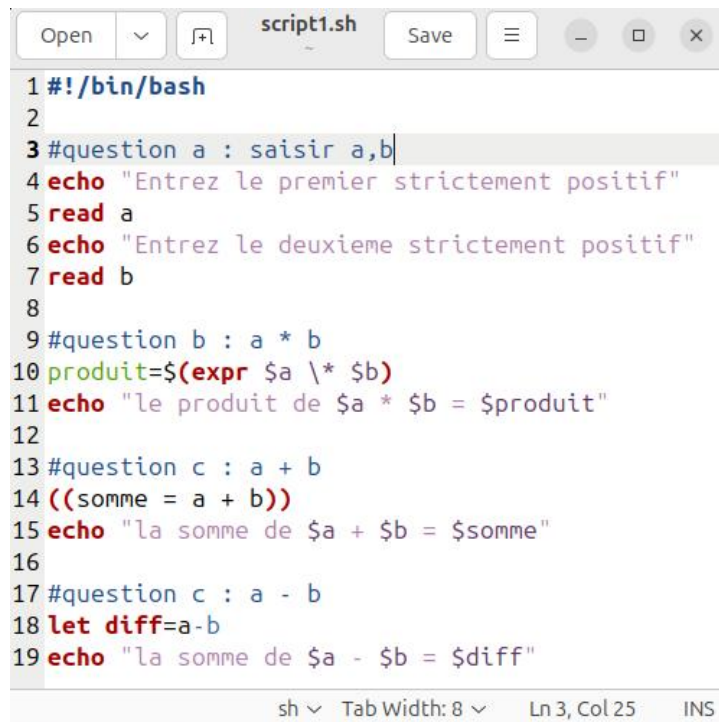
La commande :

- **grep "^d"** : filtre les lignes commençant par d (répertoires)
- **2>** : redirige les erreurs (stderr) vers un fichier
- **>** : redirige la sortie standard (stdout) vers un fichier

Compte Rendu - TDM n°3

Exercice 1 : Script shell "script1.sh"

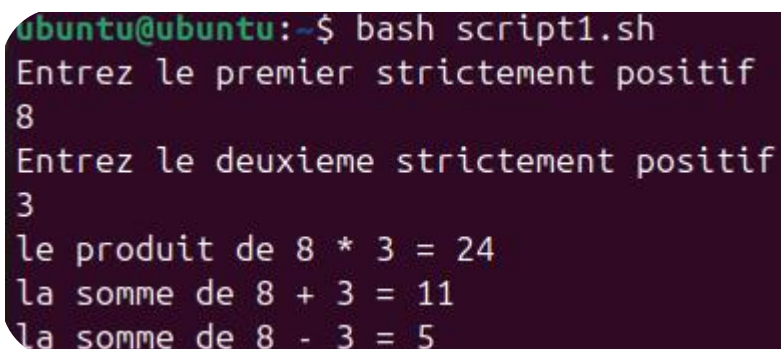
1. Développer un script shell "script1.sh" :



```
1 #!/bin/bash
2
3 #question a : saisir a,b
4 echo "Entrez le premier strictement positif"
5 read a
6 echo "Entrez le deuxieme strictement positif"
7 read b
8
9 #question b : a * b
10 produit=$(expr $a \* $b)
11 echo "le produit de $a * $b = $produit"
12
13 #question c : a + b
14 ((somme = a + b))
15 echo "la somme de $a + $b = $somme"
16
17 #question c : a - b
18 let diff=a-b
19 echo "la somme de $a - $b = $diff"
```

2. Exécuter le script shell de deux manières différentes.

Méthode 1 : Exécuter avec bash



```
ubuntu@ubuntu:~$ bash script1.sh
Entrez le premier strictement positif
8
Entrez le deuxieme strictement positif
3
le produit de 8 * 3 = 24
la somme de 8 + 3 = 11
la somme de 8 - 3 = 5
```

Méthode 2 : Rendre le script exécutable


```
ubuntu@ubuntu:~$ chmod +x script1.sh
ubuntu@ubuntu:~$ ./script1.sh
Entrez le premier strictement positif
9
Entrez le deuxieme strictement positif
2
le produit de 9 * 2 = 18
la somme de 9 + 2 = 11
la somme de 9 - 2 = 7
```

Exercice 2 : Script shell "script2.sh"

1- Développer un script shell "script2.sh" :

```
Open script2.sh Save
1#!/bin/bash
2#question a : verifier fichier.txt
3if [ ! -f "rep/fichier.txt" ]; then
4    mkdir -p rep
5    touch rep/fichier.txt
6    echo "fichier.txt cree"
7else
8    echo "fichier.txt existe deja"
9fi
10
11if [ ! -f "rep/lienPhysique.txt" ]; then
12    ln rep/fichier.txt rep/lienPhysique.txt
13    echo "lienPhysique.txt cree"
14else
15    echo "lienPhysique.txt existe deja"
16fi
17
18#question b : verifier lienSymbolique.txt
19if [ ! -L "rep/lienSymbolique.txt" ]; then
20    ln -s fichier.txt rep/lienSymbolique.txt
21    echo "lienSymbolique.txt cree"
22else
23    echo "lienSymbolique.txt existe deja"
24fi
25
26#question c : Afficher les permissions manquante
27
28for file in rep/*; do
29    echo "Fichier: $file"
30    perm=$(ls -l "$file" | cut -c2-10)
31
32    [ "${perm:1:1}" == "-" ] && echo "utilisateur : lecture manquante"
33    [ "${perm:2:1}" == "-" ] && echo "utilisateur : ecriture manquante"
34    [ "${perm:3:1}" == "-" ] && echo "utilisateur : execution manquante"
35
36    [ "${perm:4:1}" == "-" ] && echo "groupe : lecture manquante"
37    [ "${perm:5:1}" == "-" ] && echo "groupe : ecriture manquante"
38    [ "${perm:6:1}" == "-" ] && echo "groupe : execution manquante"
39
40    [ "${perm:7:1}" == "-" ] && echo "autres : lecture manquante"
41    [ "${perm:8:1}" == "-" ] && echo "autres : ecriture manquante"
42    [ "${perm:9:1}" == "-" ] && echo "autres : execution manquante"
43done
sh Tab Width: 8 Ln 42, Col 24 INS
```


2. Exécuter le script shell de deux manières différentes.

Méthode 1 : Exécuter avec bash :

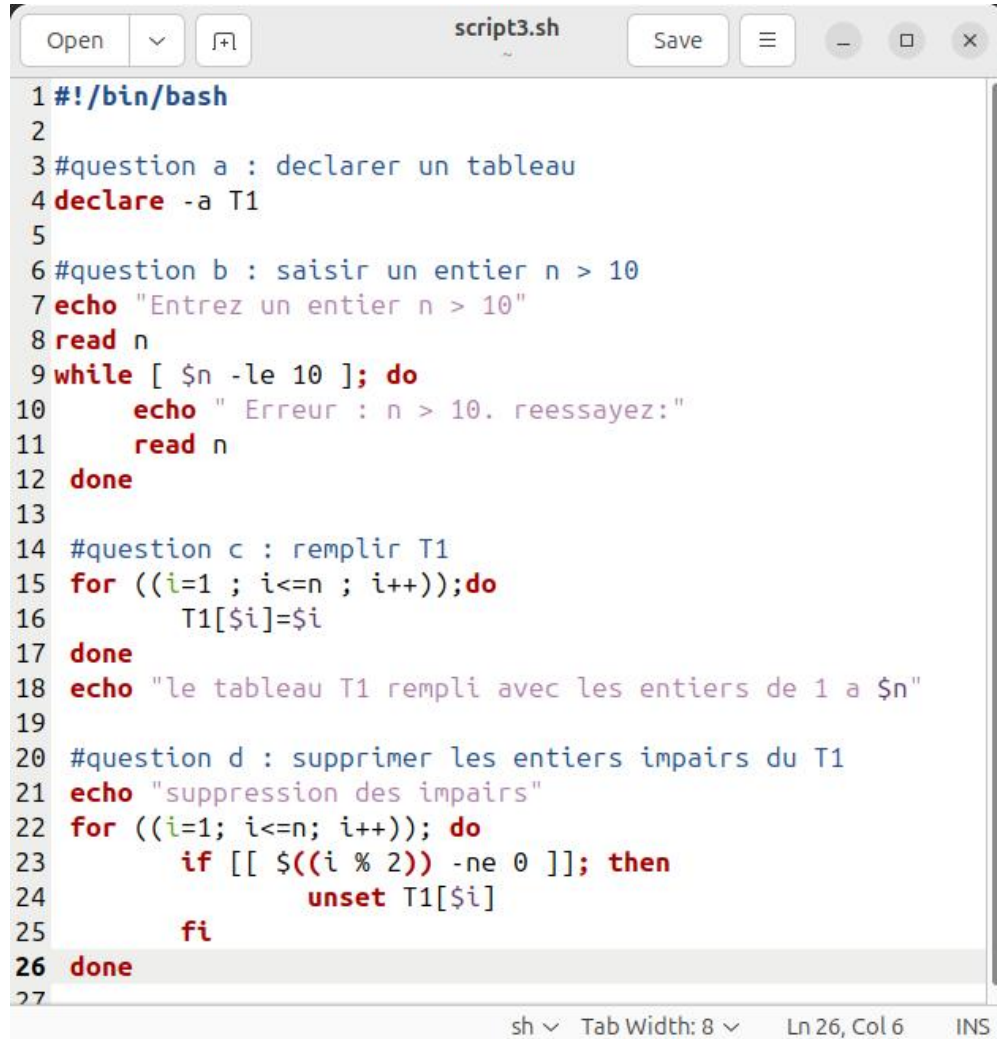
```
ubuntu@ubuntu:~$ bash script2.sh
fichier.txt existe deja
lienPhysique.txt existe deja
lienSymbolique.txt existe deja
Fichier: rep/fichier.txt
utilisateur : ecriture manquante
groupe : ecriture manquante
autres : lecture manquante
autres : ecriture manquante
Fichier: rep/lienPhysique.txt
utilisateur : ecriture manquante
groupe : ecriture manquante
autres : lecture manquante
autres : ecriture manquante
Fichier: rep/lienSymbolique.txt
ubuntu@ubuntu:~$ gedit script2.sh
```

Méthode 2 : Rendre le script exécutable :

```
ubuntu@ubuntu:~$ chmod +x script2.sh
ubuntu@ubuntu:~$ ./script2.sh
fichier.txt existe deja
lienPhysique.txt existe deja
lienSymbolique.txt existe deja
Fichier: rep/fichier.txt
utilisateur : ecriture manquante
groupe : ecriture manquante
autres : lecture manquante
autres : ecriture manquante
Fichier: rep/lienPhysique.txt
utilisateur : ecriture manquante
groupe : ecriture manquante
autres : lecture manquante
autres : ecriture manquante
Fichier: rep/lienSymbolique.txt
```

Exercice 3 : Script shell "script3.sh"

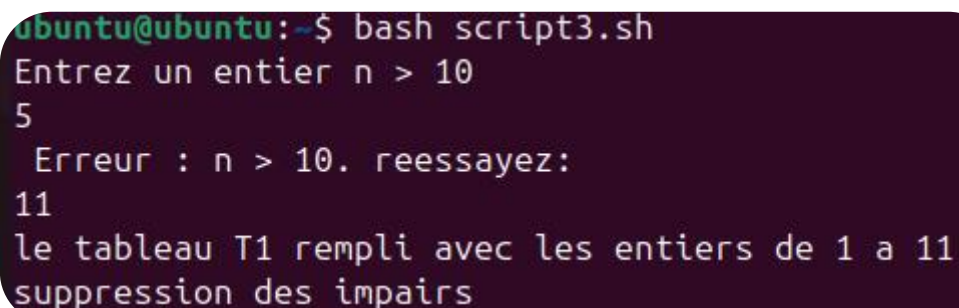
1. Développer un script shell script3.sh :



```
1#!/bin/bash
2
3#question a : declarer un tableau
4declare -a T1
5
6#question b : saisir un entier n > 10
7echo "Entrez un entier n > 10"
8read n
9while [ $n -le 10 ]; do
10     echo " Erreur : n > 10. reessayez:"
11     read n
12 done
13
14#question c : remplir T1
15for ((i=1 ; i<=n ; i++));do
16     T1[$i]=$i
17 done
18echo "le tableau T1 rempli avec les entiers de 1 a $n"
19
20#question d : supprimer les entiers impairs du T1
21echo "suppression des impairs"
22for ((i=1; i<=n; i++)); do
23     if [[ $((i % 2)) -ne 0 ]]; then
24         unset T1[$i]
25     fi
26 done
27
```

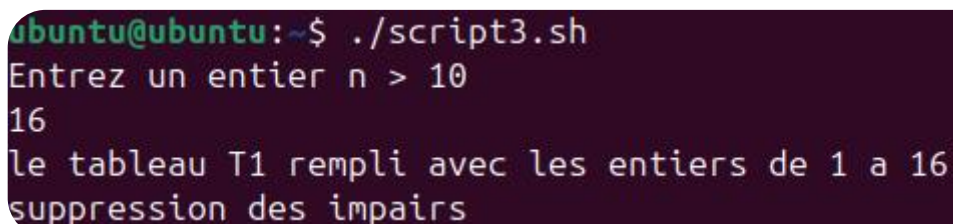
2. Exécuter le script shell de deux manières différentes.

Méthode 1 : Exécuter avec bash :



```
ubuntu@ubuntu:~$ bash script3.sh
Entrez un entier n > 10
5
 Erreur : n > 10. reessayez:
11
le tableau T1 rempli avec les entiers de 1 a 11
suppression des impairs
```

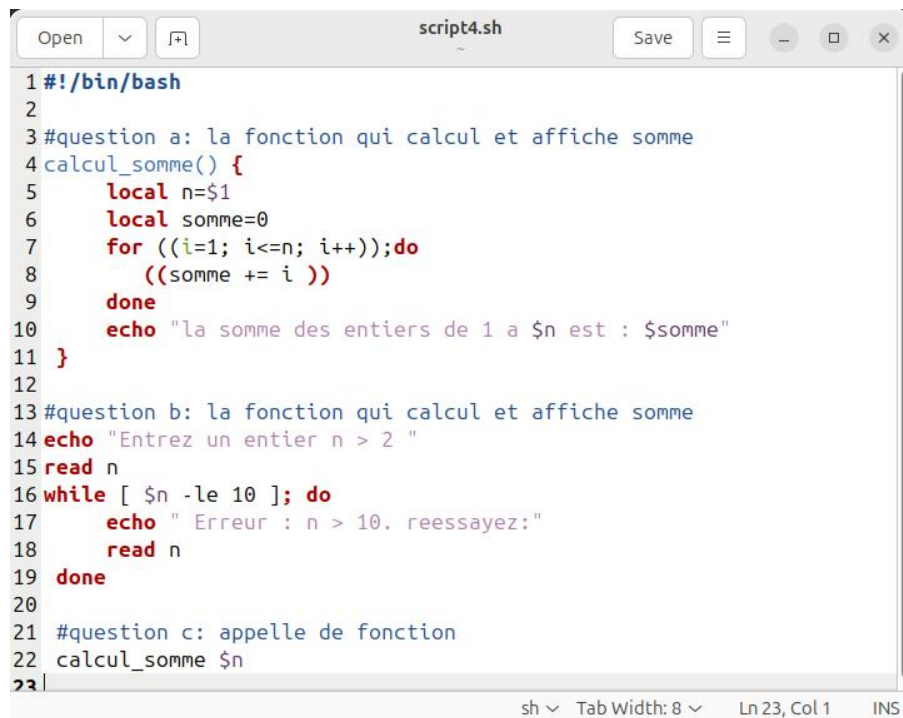
Méthode 2 : Rendre le script exécutable :



```
ubuntu@ubuntu:~$ ./script3.sh
Entrez un entier n > 10
16
le tableau T1 rempli avec les entiers de 1 a 16
suppression des impairs
```

Exercice 4 : Script shell "script4.sh" avec fonction

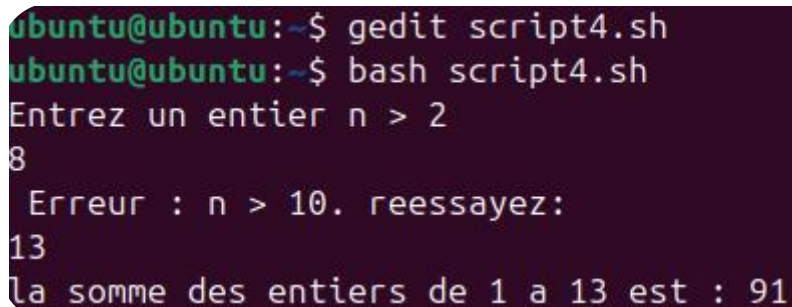
1. Développer un script shell script4.sh :



```
1#!/bin/bash
2
3#question a: la fonction qui calcul et affiche somme
4calcul_somme() {
5    local n=$1
6    local somme=0
7    for ((i=1; i<=n; i++));do
8        ((somme += i ))
9    done
10    echo "la somme des entiers de 1 a $n est : $somme"
11}
12
13#question b: la fonction qui calcul et affiche somme
14echo "Entrez un entier n > 2 "
15read n
16while [ $n -le 10 ]; do
17    echo " Erreur : n > 10. reessayez:"
18    read n
19done
20
21#question c: appelle de fonction
22calcul_somme $n
23
```

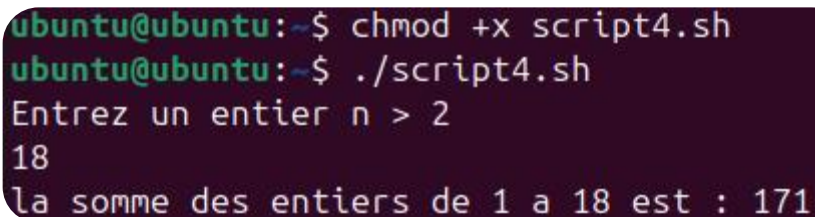
2. Exécuter le script shell de deux manières différentes.

Méthode 1 : Exécuter avec bash



```
ubuntu@ubuntu:~$ gedit script4.sh
ubuntu@ubuntu:~$ bash script4.sh
Entrez un entier n > 2
8
Erreur : n > 10. reessayez:
13
la somme des entiers de 1 a 13 est : 91
```

Méthode 2 : Rendre le script exécutable



```
ubuntu@ubuntu:~$ chmod +x script4.sh
ubuntu@ubuntu:~$ ./script4.sh
Entrez un entier n > 2
18
la somme des entiers de 1 a 18 est : 171
```